

Presentation Prep Guide

Adaptive Neural Control for Mars Rover Wheel Entrapment Recovery

Meraj Hossain Promit & Chandak Chakma | RME 4100 Final Project, April 19, 2026

Golden rule: Speak slowly, point at each diagram element as you describe it, say the number (e.g. “35%”, “200K steps”) clearly, and end every slide with one sentence about *why it matters*.

Contents

1	SLIDE 1: Title Slide	4
1.1	What is on this slide	4
1.2	What to say	4
2	SLIDE 2: The Problem That Started It All	4
2.1	What is on this slide	4
2.2	What to say	5
3	SLIDE 2b: Spirit Rover Timeline (Extended)	5
3.1	What is on this slide	5
3.2	What to say	5
4	SLIDE 3: Research Objectives	6
4.1	What is on this slide	6
4.2	What to say	6
5	SLIDE 4: Related Work & Research Gaps	6
5.1	What is on this slide	6
5.2	What to say	7
6	SLIDE 4b: Comparison with Bi & Ding 2026	7
6.1	What is on this slide	7
6.2	What to say	8
6.2.1	Why GRU not LSTM? (This will definitely be asked!)	8
7	SLIDE 5b: Escape Policy Generation Diagram (Fig. 2)	8
7.1	What is on this slide	8
7.2	What to say	9
8	SLIDE 5c: Simulation Framework (Fig. 3)	9
8.1	What is on this slide	9
8.2	What to say	10
9	SLIDE 5d: Robot Platform — AAU 6-Wheel Rover	10
9.1	What is on this slide	10
9.2	What to say	10

10 SLIDE 6: Physics Simulation Stack	11
10.1 What is on this slide	11
10.2 What to say	11
10.2.1 What is MPM? (Plain language)	12
11 SLIDE 7: RL Formulation — POMDP & Reward Design	12
11.1 What is on this slide	12
11.2 Understanding the PPO equations — explained simply	12
11.2.1 The POMDP objective	12
11.2.2 Why POMDP and not MDP?	13
11.2.3 The PPO clipped objective	13
11.2.4 GAE and the advantage	13
11.3 The Reward Function (7 components)	14
11.4 What to say	14
12 SLIDE 7b: GRU-PPO Policy Network Architecture	15
12.1 What is on this slide	15
12.2 What to say	15
12.2.1 Why a stochastic policy?	16
12.2.2 What is GRU? (Simple explanation)	16
13 SLIDE 8: Dual-Sensor Entrapment Detection	16
13.1 What is on this slide	16
13.2 What to say	17
13.2.1 What is CNN-GRU? (For Phase 1)	17
14 SLIDE 9: Curriculum Learning & Training Pipeline	18
14.1 What is on this slide	18
14.1.1 curriculum_milestones.png — what this plot shows	18
14.2 What to say	18
15 SLIDE 9b: DRL Training Loop Block Diagram	19
15.1 What is on this slide	19
15.2 What to say	19
16 SLIDE 10: Training Overview — training_overview.png	19
16.1 What is on this slide	19
16.1.1 training_overview.png — what the panels show	20
16.2 What to say	20
17 SLIDE 11: Escape Progress & Curriculum — escape_rate.png	20
17.0.1 escape_rate.png — what it shows	20
17.0.2 curriculum_milestones.png — what it shows	21
17.1 What to say	21
18 SLIDE 12: Loss & Policy Metrics	21
18.0.1 losses.png — what it shows	21
18.0.2 policy_metrics.png — what it shows	21
18.1 What to say	22

19 SLIDE 12b: Reward Breakdown — reward_breakdown.png	22
19.0.1 reward_breakdown.png — what it shows	22
19.0.2 detection_flags.png — what it shows	22
20 SLIDE 13: Escape Behaviour Demo — env_metrics.png	22
20.0.1 env_metrics.png — what it shows	22
20.1 What to say	23
20.1.1 Emergent strategies explained	23
21 SLIDE 14: Training Results Summary	23
21.0.1 summary_table.png — what it shows	23
21.1 What to say	23
22 SLIDE 16: Future Work Roadmap	23
22.1 What is on this slide	23
22.2 What to say	24
23 SLIDE 17: Contributions	24
23.1 What to say	24
24 EXPECTED QUESTIONS & MODEL ANSWERS	25
25 QUICK-REFERENCE NUMBERS CARD	29
26 PRESENTATION TIMING GUIDE	30

SLIDE 1: Title Slide

What is on this slide

- Full project title: *Adaptive Neural Control for Mars Rover Wheel Entrapment Recovery in Granular Terrain*
- Your names and supervisor/co-supervisor names
- University of Dhaka logo (top right)
- Decorative Mars terrain silhouette at the bottom (dune ridges and wheel tracks), a faint Mars circle on the right, and small star dots on the background

What to say

“Good afternoon everyone. I am Meraj Hossain Promit, and my co-presenter is Chandak Chakma. Our project is titled Adaptive Neural Control for Mars Rover Wheel Entrapment Recovery in Granular Terrain, submitted for RME 4100 under the supervision of Md. Jubair Ahmed Sourav and co-supervisor Dr. Sejuti Rahman. Thank you for being here.”

SLIDE 2: The Problem That Started It All

What is on this slide

- **Left panel:** A tcolorbox listing 5 bullet points about the Spirit rover incident at “Troy” (May 2009). Key facts: 5 years of exploration before entrapment, 8 months of recovery attempts, 4–24 minute signal latency, no autonomous escape, mission ended permanently.
- **Italic quote box:** *“What if the rover could have saved itself?”* — this is the emotional hook of your whole project.
- **Right panel:** A vertical timeline drawn with TikZ. Six coloured dots going from top to bottom:
 1. Green dot — Jan 2004: Spirit Lands on Mars
 2. Teal dot — Apr 2009: 5 Years Successful Exploration
 3. Red dot — May 2009: **Entrapped at “Troy”**
 4. Orange dot — Jun–Dec 2009: 8-Month Rescue Attempt
 5. Gold dot — Jan 2010: Stationary Science Mode
 6. Gray dot — May 2011: Contact Lost, Mission Ended
- **Bottom stat box:** 150M km — Earth–Mars distance during entrapment

What to say

“The motivation for our work comes from a real NASA tragedy. In May 2009, after 5 years of successful exploration, the Spirit rover drove into a patch of soft, sulfate-rich regolith at a location called Troy. The wheels broke through a thin crust and sank into loose soil. NASA spent 8 months sending increasingly desperate recovery commands from Earth — but with a one-way signal delay of 4 to 24 minutes, and no autonomous recovery capability on the rover itself, every attempt failed. Spirit never moved again.

You can see this story on the timeline on the right. The red dot marks the moment of entrapment. Everything below it is the story of a machine that could not save itself, 150 million kilometres from home.

That one question — what if it could have saved itself? — is the entire motivation for our research.”

Key numbers:

- Spirit: 5 years of exploration before entrapment
- 8 months of recovery attempts, all failed
- 4–24 minute one-way signal delay
- 150 million km Earth–Mars distance

SLIDE 2b: Spirit Rover Timeline (Extended)

What is on this slide

- A horizontal flowchart with 6 boxes connected by arrows: Spirit Landed → 5 Years Exploration → **Entrapped at Troy** (highlighted in red) → 8-Month Rescue Attempt (gold highlight) → Stationary Science → Contact Lost
- Below the boxes: a time axis showing dates (Jan 2004 to May 2011)
- A caption text: “Spirit’s wheels broke through a thin crusty surface into soft sulfate-rich soil. Despite 8 months of ground-based testing and careful extraction commands, the rover remained permanently trapped, ending its mobile mission.”
- A red highlight box at the bottom: “Key lesson: Spirit had *no onboard capability* to detect or respond to entrapment autonomously. All recovery attempts were commanded from Earth with 8–40 min round-trip delay. **Our work builds the autonomy Spirit never had.**”

What to say

“This slide zooms in on the full 7-year story. The critical gap you can see here is between the orange box — 8 months of rescue attempts — and the final gray box — contact lost. All recovery commands had to travel 300 million kilometres round-trip, with up to 40 minutes of latency. There was simply no way to react in real time.

Our work directly addresses this gap: we want the rover to detect entrapment by itself, on the rover, and execute a recovery policy without waiting for a signal from Earth.”

SLIDE 3: Research Objectives

What is on this slide

- Five objective cards (tcolorboxes), arranged in a two-column layout:
 1. **Obj 1: Realistic Simulation** — couple Newton MPM granular physics with Isaac Lab rigid-body solver under Martian gravity (3.72 m/s^2)
 2. **Obj 2: Entrapment Detection** — dual-sensor scheme: slip-ratio monitoring + drive-torque anomaly, no GPS
 3. **Obj 3: Recovery Policy** — recurrent RL policy (PPO + GRU) discovering rocking, steering pivots, differential drive
 4. **Obj 4: Curriculum Learning** — progressive sinkage $12 \text{ cm} \rightarrow 22 \text{ cm}$ across 64 parallel environments
 5. **Obj 5: Sim-to-Sim & Sim-to-Real** — cross-simulator validation; ONNX export to Raspberry Pi 5 at 10 Hz
- A green contribution box: “First DRL escape system for a 6-wheel rocker-bogie rover in physics-accurate granular simulation”

What to say

“Our research has five concrete objectives. First, we need a realistic simulation — not a simple flat floor, but real granular soil physics under Martian gravity of 3.72 m/s^2 .

Second, we need the rover to detect entrapment on its own, using only wheel slip and motor torque — sensors that any rover already has.

Third, we train a neural network policy using PPO with a GRU — a recurrent network — to discover escape strategies through trial and error.

Fourth, we use curriculum learning: we start with shallow sinkage and gradually increase it, so the agent builds competence progressively.

Fifth, our goal is to eventually deploy this on a Raspberry Pi 5 through ONNX export, and to validate across different simulators.

The key novelty — highlighted here in green — is that this is the first time anyone has combined a 6-wheel rocker-bogie rover, physics-accurate granular soil simulation, and deep reinforcement learning in the same system.”

Key numbers:

- Martian gravity: 3.72 m/s^2 (vs Earth 9.81 m/s^2)
- Sinkage range: $12 \text{ cm} \rightarrow 22 \text{ cm}$
- 64 parallel training environments
- Target deployment: Raspberry Pi 5, 10 Hz

SLIDE 4: Related Work & Research Gaps

What is on this slide

- A comparison table with 5 prior works (rows) and columns: Robot, Key Limitation, Terrain, Gravity, Escape%

- **Bi & Ding 2026:** 4-wheel SSMR, rigid terrain, no granular physics, needs expert demos, Earth gravity, 90% escape
- **Mortensen & Bøgh 2024:** AAU 6-wheel, navigation only, no entrapment task, rigid terrain, Mars gravity
- **Inotsume 2020:** detection only, no recovery policy, Bekker terrain, Mars gravity
- **Jain 2019:** discrete action space, approximate terrain, Mars gravity
- **Fankhauser 2018:** legged robot (ANYmal), different failure mode, Earth gravity
- **Ours** (bottom row, highlighted): 6-wheel RB, MPM granular, Mars gravity, ~35% escape (early training)
- Three gap boxes: Gap 1 (no 6-wheel+MPM+DRL), Gap 2 (no granular sinkage in DRL), Gap 3 (no 10D per-wheel control)
- A note about RLRoverLab (Mortensen & Bøgh, ISPaRo 2024) as a dependency we built upon

What to say

“Looking at the state of the art, we find several gaps. The closest work in performance terms is Bi & Ding 2026, which achieves 90% escape rate — but on rigid terrain, on Earth, with expert demonstration data, and with a simpler 4-wheel robot.

The only prior work using the same AAU rover is Mortensen and Bøgh, but they only study navigation — no entrapment scenario at all.

What nobody has done before is combine all three ingredients at once: a 6-wheel rocker-bogie rover, physics-accurate granular terrain modelling with MPM, and deep reinforcement learning. That is our contribution.

Our current result of 35% escape rate is lower than Bi & Ding, but we used only 200K training steps versus their 4 million, and our physics are vastly more realistic.”

Key numbers:

- Bi & Ding: 90% escape, 4M steps, rigid terrain, Earth gravity
- Ours: ~35% escape, 200K steps, MPM granular, Mars gravity
- 3 research gaps all filled by our work simultaneously

SLIDE 4b: Comparison with Bi & Ding 2026

What is on this slide

- A detailed comparison table: Ours vs Bi & Ding across 10 aspects (robot, terrain, gravity, observation size, action size, policy, training steps, reward type, escape rate, real-world validation)
- A red warning box: their edge — 20× more training, real-world validated, GAIL+expert demos
- A green box: our edge — physics-accurate MPM soil, Mars gravity, 5× richer action space, no expert data
- **Right panel:** A card titled “Why GRU not LSTM?” with two bullet lists

What to say

“This slide directly compares us against the best published system. The differences tell the story of our design choices.

Their robot has 2 action dimensions — just left and right drive speed. Ours has 10 — 6 independent wheel speeds plus 4 steering angles. That is a 5-times richer action space, which is necessary for a rocker-bogie suspension.

They used GAIL — Generative Adversarial Imitation Learning — which requires an expert to demonstrate correct escape behaviour. We cannot do that for Martian regolith, so we designed our reward function entirely from physical principles, with no expert demonstrations.

The most important difference is the terrain model. Rigid terrain cannot model the sinkage, bulldozing, or particle redistribution that actually traps rovers. Our MPM simulation does all of that.”

Why GRU not LSTM? (This will definitely be asked!)

- **GRU has 2 gates** (reset gate, update gate); LSTM has 3 gates (input, forget, output) plus a separate cell state
- **~30% fewer parameters** than an equivalent LSTM, which means faster inference — critical for running on a Raspberry Pi 5 (GRU <5 ms vs LSTM ~7 ms per inference step)
- **No separate cell state**: GRU merges the hidden state and cell state, simpler gradient flow (BPTT = backpropagation through time)
- **Our episodes are short**: 200–600 steps at 25 Hz = 8–24 seconds of physical time, with sequence length 16. GRU performs equivalently to LSTM on sequences shorter than ~5 seconds; for longer sequences LSTM can have an advantage, but not here
- PPO’s clipping already stabilises gradient flow, so LSTM’s cell state is doubly redundant

SLIDE 5b: Escape Policy Generation Diagram (Fig. 2)

What is on this slide

This is a large TikZ diagram showing the full training and deployment pipeline. It has **three vertical zones**:

1. **Top zone — Simulation environment**: Two boxes (MPM Granular Regolith Simulation + MuJoCo Rigid Body Dynamics) inside a Newton Physics Simulation box. Below them: Randomized Sinkage Depth (Curriculum Learning).
2. **Middle zone — Training loop**: An Entrapment State Identification block on the right. A Shaped Reward Module (R_H) and a Curriculum Bonus Module feed into a plus-node, which feeds the reward into the DRL block (PPO-GRU, 64 parallel envs). The DRL block produces the trained Escape Action Generation Model (blue box).

3. **Bottom zone — Deployment:** The trained model is transferred. At runtime: sensor state $\mathbf{s}$ is concatenated with entrapment flag s_{entrap} , fed to the Escape Action Generation Model, which outputs action $\mathbf{a}$, which drives the Mars Rover (sim or physical). The rover's new state feeds back to the Entrapment State Identification block, closing the loop.

Arrow colours: red = DRL training dataflow, blue = model transfer, black = standard data.

What to say

“This diagram shows the complete system pipeline in two halves. The top half is training, the bottom half is deployment.

During training, the Newton physics engine gives us two coupled components: an MPM granular sand simulation on the left, and a MuJoCo rigid body simulation of the rover on the right. These two communicate two-ways: the rover wheels push sand particles, and the sand pushes back on the wheels.

From this simulation we read out three types of state signals: motor state (torques, velocities), motion state (IMU, displacement), and the entrapment state flag. The entrapment flag is computed by the Entrapment State Identification block.

These signals go into the PPO-GRU training loop, which receives a shaped reward signal with 7 components. After training, the policy is packaged into the Escape Action Generation Model.

At deployment — shown in the bottom zone — the same loop runs but now on the physical rover or a target simulator. The rover's sensors produce state signals, the entrapment detector flags whether we are stuck, and the trained model outputs the 10-dimensional action to the motor controllers.”

SLIDE 5c: Simulation Framework (Fig. 3)

What is on this slide

Another large TikZ diagram showing the software architecture layers:

- An outer “Environment” rectangle
- Inside: a “Newton MPM + MuJoCo simulation” box containing 6 rows (one per wheel: FL, ML, RL, FR, MR, RR), each with two sub-boxes: MPM particle interaction → Wheel contact force → arrow to MuJoCo Rigid Body Dynamics
- Below the simulation box: Randomized Sinkage Depth Generation (MPM particle bed, curriculum-driven)
- Below the environment: Mars Rover Control Policy box, with three internal boxes: Random Policy → Curriculum PPO Policy → Reward-shaped GRU-PPO Policy
- A left feedback loop: Action $\mathbf{a} = [\omega_{\text{drive}}, \theta_{\text{steer}}]$ going up to the environment
- A right feedback loop: State $\mathbf{s} = \{\mathbf{s}_{\text{motor}}, \mathbf{s}_{\text{motion}}, \mathbf{s}_{\text{entrap}}\}$ going down to the policy

What to say

“This figure shows the same system but from a software architecture perspective. The key idea is the two-way coupling in the centre.

For each of the six wheels — front left, middle left, rear left, and their right counterparts — we simulate two things independently: first, how MPM particles (the sand) interact with the wheel, and second, the resulting contact force. That contact force goes into MuJoCo’s rigid body solver, which computes how the rover body moves.

At the bottom is the curriculum: the sand bed’s initial depth changes progressively during training. The policy box at the bottom shows three stages of policy development: random exploration, curriculum PPO, and finally the reward-shaped GRU-PPO policy that is our final system.

The two external arrows show the control loop: actions go in from the left, observations come out on the right.”

SLIDE 5d: Robot Platform — AAU 6-Wheel Rover

What is on this slide

- **Top-left image:** Screenshot of the AAU Mars Rover 3D USD asset as it appears in the RLRoverLab framework. A detailed 6-wheel rocker-bogie rover model.
- **Bottom-left image:** The rover sitting on the MPM regolith bed inside the Newton simulation (the “entrapment” image). You can see the rover partially sunk into the sand.
- **Right panels:** Information about RLRoverLab (open-source, by Magni et al.), our modifications (adding Newton MPM coupling), and a joint configuration table:

Joint type	Count	Range
Drive (continuous)	6	± 6 rad/s
Steer (revolute)	4	± 0.6 rad
Passive (rocker/diff.)	several	Free

- Bottom highlight box: “10D action space: 6 independent drive ω + 4 steer θ , the richest per-wheel control in any entrapment DRL work”

What to say

“The robot we use is the AAU Mars Rover, a 6-wheel rocker-bogie platform from the RLRoverLab open-source framework. The image on the top left shows the full 3D model. The image below shows what it looks like inside our Newton simulation, partially sunk into the granular regolith bed.

The rover has three types of joints. Six drive joints spin the wheels continuously at up to 6 radians per second. Four steer joints turn the corner wheels up to 0.6 radians. The remaining joints are passive: they form the rocker-bogie suspension linkage that allows individual wheels to move independently over uneven terrain.

This gives us a 10-dimensional action space — by far the richest control input in any rover entrapment recovery study. Most prior work uses a 2-wheel model with one action dimension per side.”

SLIDE 6: Physics Simulation Stack

What is on this slide

- **Left:** Two tcolorboxes describing MPM and the sand bed configuration. Key MPM bullet points: hybrid Eulerian-Lagrangian, Drucker-Prager yield surface for granular flow, 38,400 particles per environment.
- **Right:** A layered stack diagram (Fig. 7) showing 5 layers from top to bottom:
 1. **RL Agent Layer** (blue): PPO-GRU Policy at 25 Hz — sends actions $\mathbf{a}_t$ down
 2. **Environment Layer** (orange): Isaac Lab DirectRLEnv at 50 Hz — sends observations $\mathbf{o}_t, r_t$ up
 3. **Rigid-Body Solver** (green): MuJoCo CPU, 4 substeps
 4. **Granular Solver** (red): SolverImplicitMPM, 38K particles, $\mu = 0.9$
 5. **Ground Plane** (purple): Newton callback
- Between the rigid-body and granular layers: two bold red arrows showing two-way coupling — “wheel SDF” going down, “sand impulse” going up

What to say

“The physics engine is the most novel part of our infrastructure. Let me explain the stack from bottom to top.

At the very bottom is the ground plane, implemented using a Newton callback.

Above that is the granular solver — this is Newton’s SolverImplicitMPM. MPM stands for Material Point Method, a technique from computational mechanics that models soil as a continuum using hybrid particle-and-grid representation. Each of our 64 environments has 38,400 particles. The Drucker-Prager yield surface tells the simulation when sand flows plastically — exactly what happens when a wheel sinks in.

Above MPM is MuJoCo handling the rigid body dynamics of the rover itself.

The critical detail is the two red arrows between these two layers: the rover wheels’ shape (technically their Signed Distance Function) pushes sand particles down, and the displaced sand pushes back up on the rover body. This two-way coupling is what makes sinkage physically realistic.

Above that, Isaac Lab’s environment layer runs at 50 Hz and handles the reward computation and episode resets. The PPO-GRU policy runs at 25 Hz — every two physics steps — because the robot’s mechanical timescale is slower than the physics.”

Key numbers:

- 38,400 MPM particles per environment
- Physics at 50 Hz, policy at 25 Hz
- Two-way coupling: wheel SDF $\rightarrow$ sand, sand impulse $\rightarrow$ rover
- Martian gravity: $g = 3.72 \text{ m/s}^2$
- Sand friction $\mu = 0.9$, bed size $2 \times 2 \times 0.25 \text{ m}$

What is MPM? (Plain language)

- Classical methods split into two camps: particle methods (track individual particles — accurate but slow) and grid methods (fixed mesh — fast but bad at large deformation like flowing sand)
- MPM uses **both**: particles carry mass, momentum, and deformation history; a background grid is used to compute forces each step, then the answer is transferred back to particles
- The grid is cleared and rebuilt every timestep — so it handles arbitrary deformation (sand pouring, sinkage) naturally
- The Drucker-Prager model is a stress criterion: it says “below this stress, the sand behaves like a solid; above it, it flows like a liquid” — exactly how real sand works

SLIDE 7: RL Formulation — POMDP & Reward Design

What is on this slide

- **Left column:**
 - POMDP objective equation: $J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t) \right]$, $\gamma = 0.99$
 - Explanation of partial observability: rover cannot sense per-wheel sinkage depth or terrain composition directly
 - PPO clipped objective: $\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$
- **Right column:** Reward function table with 7 components

Understanding the PPO equations — explained simply

The POMDP objective

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t) \right]$$

- π_θ = our neural network policy with parameters θ (the weights). It maps observations to actions.
- τ = a *trajectory*, i.e. one complete episode: a sequence of (state, action, reward, next state, ...) tuples
- $\mathbb{E}_{\tau \sim \pi_\theta}[\cdot]$ = “on average, over all episodes generated by our current policy”
- $R(s_t, a_t)$ = the reward we get at timestep t for being in state s_t and taking action a_t
- $\gamma^t = 0.99^t$ = the discount factor. A reward tomorrow is worth 99% of a reward today. A reward 100 steps away is worth $0.99^{100} \approx 36.6\%$ of an immediate reward. This makes the agent prefer escaping sooner rather than later.
- $J(\pi_\theta)$ = the total expected discounted return. We want to find θ that maximises this.

Why POMDP and not MDP?

- In a Markov Decision Process (MDP), the agent can observe the full state — everything relevant about the world.
- Here, the rover **cannot** directly observe: how deep each wheel is sunk, the exact soil composition, or whether it has truly escaped or just temporarily moved forward.
- This is a Partially Observable MDP (POMDP). The agent only sees a 29-dimensional observation vector (wheel velocities, slip ratios, IMU, etc.) — not the full physics state.
- The GRU hidden state h_t is the agent’s memory. It looks at the *history* of observations and implicitly infers things it cannot directly see, like “my wheel was slipping for the past 15 steps, I am probably still sunk.”

The PPO clipped objective

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ = the probability ratio. It measures how much the new policy differs from the old policy for the same action.
 - If $r_t > 1$: new policy finds this action more likely than before
 - If $r_t < 1$: new policy finds this action less likely than before
- $\hat{A}_t$ = advantage estimate. Positive if action a_t was better than the average; negative if it was worse.
- The naive policy gradient would just multiply $r_t \cdot \hat{A}_t$ and push r_t as large or small as possible. This is unstable.
- PPO adds a **clip**: $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$ forces r_t to stay between 0.8 and 1.2 (with $\epsilon = 0.2$). Any update outside this band is ignored.
- The min means we always take the more pessimistic of the two estimates, preventing overconfident policy updates.
- **In plain English:** PPO says “improve the policy, but don’t change it too drastically in one step.” This makes training stable.

GAE and the advantage

- $\hat{A}_t$ is computed by Generalised Advantage Estimation (GAE) with $\lambda = 0.95$
- The critic network estimates $V(s_t)$ = the expected future return from state s_t
- The advantage is $\hat{A}_t = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}$, where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (TD error)
- $\lambda = 0.95$ means we use mostly actual rewards, with a small bias from the value function — a balance between variance (pure returns) and bias (pure value function)

The Reward Function (7 components)

$$R_t = r_{\text{progress}} + r_{\text{escape}} + r_{\text{rocking}} - p_{\text{slip}} - p_{\text{tilt}} - p_{\text{smooth}} - p_{\text{abnormal}}$$

- r_{progress} (+1.5) Forward velocity reward. Each timestep, reward = $1.5 \times v_x \times dt$. This motivates moving forward, but when the rover is trapped ($v_x \approx 0$), this term contributes almost nothing. It does not fire backwards (penalising retreat).
- r_{escape} (+20) Sparse milestone reward. If the rover moves 0.5 m: +4 reward. If it reaches 1.0 m: +8. If it reaches 1.5 m: +20 (scaled by a time bonus). This is the dominant signal and teaches the agent the fundamental goal: get distance from the entrapment point.
- r_{rocking} (+2.0) Rocking reward: $2.0 \times \Delta v_x \times f_{\text{ent}} \times dt$, active only when the entrapment flag $f_{\text{ent}} = 1$. It rewards *changing* forward velocity while trapped — incentivising the back-and-forth rocking motion that packs sand beneath wheels.
- p_{slip} (-0.5) Slip penalty: $0.5 \times \bar{\sigma}_{\text{slip}} \times (1 - f_{\text{ent}}) \times dt$. Penalises excessive wheel spin *only after escape*. During entrapment, some slip is inevitable and necessary, so the $(1 - f_{\text{ent}})$ factor turns this off.
- p_{tilt} (-0.3) Tilt penalty: $0.3 \times \|\omega_{xy}\|_2 \times dt$. Penalises rolling and pitching, keeping the rover stable.
- p_{smooth} (-0.05) Smoothness penalty: $0.05 \times \|\Delta \mathbf{a}\|_2 \times dt$. Penalises large action changes between timesteps. Encourages smooth, physically plausible control.
- p_{abnormal} (-0.3) Abnormal motion penalty: $0.3 \times (-v_x)^+ \times f_{\text{anom}} \times dt$. Penalises backward motion *during a torque anomaly*. Currently fires $< 1\%$ of the time because the dual-condition gate (torque $> 85\%$ AND $\Delta\tau > 0.10$ for 20 steps) is rarely triggered — a known limitation.

Key numbers:

- Escape reward weight 20 dominates; it is the most important learning signal
- Discount $\gamma = 0.99$, GAE $\lambda = 0.95$, PPO clip $\epsilon = 0.2$
- Entrapment flag preset to 1 at episode start (rover spawns buried)
- Slip penalty disabled during entrapment to allow necessary wheel spin

What to say

“We formulate the problem as a Partially Observable Markov Decision Process, or POMDP. The key word is partial: the rover cannot directly observe how deep its wheels are sunk, or the exact soil composition. It only sees 29 sensor readings. The GRU memory network compensates for this by tracking history.

The objective is to find a policy that maximises expected discounted reward. The discount factor 0.99 means the agent slightly prefers escaping sooner.

We use PPO — Proximal Policy Optimisation. The clipped objective is the core innovation of PPO: it prevents the policy from changing too drastically in any single update, which keeps training stable.

The reward function has 7 terms. The most important is the escape milestone reward, weighted at 20. This sparse signal fires when the rover travels 0.5, 1.0, or 1.5 metres

from its starting point. The rocking reward at weight 2 specifically incentivises the back-and-forth motion that physically dislodges wheels from granular traps.”

SLIDE 7b: GRU-PPO Policy Network Architecture

What is on this slide

A large TikZ diagram of the neural network. Reading left to right:

1. **Input layer** (8 dots shown schematically): $\mathbf{s}_t \in \mathbb{R}^{29}$. Below: “6 vel + 6 slip + 4 steer + 3 IMU + 1 grav + 6 torque + 2 flags + 1 disp”
2. **First FC layer**: 7 neurons shown (128 units actual), labelled “FC”. Fully connected from all 29 inputs.
3. **GRU cell**: A blue rectangle labelled “GRU / 256 hidden”. It receives the 128-dimensional FC output *and* the previous hidden state $\mathbf{h}_{t-1}$. It outputs the new hidden state $\mathbf{h}_t$ (256 dimensions).
4. **Second FC layer**: 6 neurons shown (64 units actual). Fully connected from the GRU output.
5. **Output heads** (two separate sets):
 - **Drive outputs (6D)**: 6 orange nodes $\mu_{FL}, \mu_{ML}, \mu_{RL}, \mu_{FR}, \mu_{MR}, \mu_{RR}$ (mean wheel speeds), plus 1 red node $\log \sigma_{\text{drive}}$. These combine to produce $\mathbf{a}_{\text{drive}} \in \mathbb{R}^6$.
 - **Steer outputs (4D)**: 4 orange nodes $\mu_{FL}, \mu_{FR}, \mu_{ML}, \mu_{MR}$ (mean steer angles), plus 1 red node $\log \sigma_{\text{steer}}$. These combine to produce $\mathbf{a}_{\text{steer}} \in \mathbb{R}^4$.
6. A legend box bottom-centre: μ = mean of Gaussian policy; $\log \sigma$ = log std deviation; $\mathbf{a}$ = sampled action

What to say

“This diagram shows exactly how our neural network is structured. The data flows from left to right.

On the far left, the 29-dimensional observation vector enters. These 29 numbers represent: 6 wheel velocities, 6 slip ratios, 4 steering positions, 3 IMU accelerations, 1 gravity reading, 6 drive torques, 1 entrapment flag, 1 torque anomaly flag, and 1 normalised displacement.

This is passed through a first fully-connected layer of 128 neurons, which extracts low-level features.

The central element is the GRU cell with 256 hidden units. The GRU receives two inputs simultaneously: the 128-dimensional feature vector and its own hidden state from the previous timestep, $\mathbf{h}_{t-1}$. This is how the network has memory. After processing, it produces the new hidden state $\mathbf{h}_t$.

The hidden state goes through another fully-connected layer of 64 units, and then splits into two output heads.

For drive control, the network outputs a mean μ for each of the 6 wheels, plus a shared log-standard-deviation. Together these define a Gaussian probability distribution over

wheel speeds. The actual action is a sample from this distribution during training, which drives exploration.

Similarly for steering: 4 means and a shared log-standard-deviation define the steering action distribution.

This is what we call a *stochastic policy*: rather than outputting one fixed action, it outputs a probability distribution. During evaluation, we use the mean directly.”

Why a stochastic policy?

- In PPO, the policy must be differentiable and must produce action probabilities (not just actions)
- During training, sampling from the distribution provides **exploration**: the rover tries different escape strategies by chance
- The log-probability of the action taken is used in the PPO ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$
- The shared log σ (one per head, not per output neuron) keeps the architecture compact while still allowing the policy to modulate its own confidence

What is GRU? (Simple explanation)

- A GRU (Gated Recurrent Unit) is a type of recurrent neural network cell
- It has two gates:
 - **Reset gate** r_t : decides how much of the past hidden state to forget before computing the candidate new hidden state. If $r_t \approx 0$, the new candidate ignores the past entirely.
 - **Update gate** z_t : decides how much of the old hidden state to keep vs. replace with the new candidate. If $z_t \approx 1$, the hidden state is barely updated (long memory). If $z_t \approx 0$, it is almost fully replaced.
- The GRU equations:

$$\begin{aligned} r_t &= \sigma(W_r[h_{t-1}, x_t]) \\ z_t &= \sigma(W_z[h_{t-1}, x_t]) \\ \tilde{h}_t &= \tanh(W[r_t \odot h_{t-1}, x_t]) \quad (\text{candidate}) \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \end{aligned}$$

- σ = sigmoid (squashes to 0–1); $\tanh$ = squashes to -1 to 1 ; $\odot$ = element-wise multiply
- In our setting, x_t is the 128-dim FC output; h_{t-1} is the 256-dim hidden state from last step

SLIDE 8: Dual-Sensor Entrapment Detection

What is on this slide

- **Left column**: Two boxes explaining the formal entrapment criterion and the two detection mechanisms

- Entrapment flag: $f_{\text{ent}} = 1$ if $|v_x| < 0.15$ m/s AND $\bar{\sigma}_{\text{slip}} > 0.4$ for ≥ 15 consecutive steps (0.6 seconds)
- Torque anomaly: $f_{\text{anom}} = 1$ if $|\overline{\tau_i/\tau_{\text{max}}}| > 0.85$ AND $\Delta\bar{\tau} > 0.10$ for ≥ 20 steps
- **Right column:** The Dual-Brain Onboard Architecture diagram (Fig. 11). This shows the onboard computer (Raspberry Pi 5 dashed rectangle) containing:
 - Three sensor boxes at top: IMU, Encoders, Goal/GPS
 - CNN-GRU Entrapment Detector (Phase 1): outputs $P(\text{normal})$, $P(\text{sinking})$, $P(\text{entrapped})$
 - A diamond decision node: “Entrapped?”
 - Green arrow left: Nav Brain (RLRoverLab, point-to-point) if No
 - Red arrow right: Escape Brain (our GRU-PPO, entrap recovery) if Yes
 - Both feed into Motor Commands (drive $\times 6$ + steer $\times 4$)
 - A dashed arrow from Escape Brain back to the decision node (“escaped” feedback)

What to say

“Entrapment detection works on two complementary signals.

The primary signal is slip ratio. Slip ratio measures the difference between how fast the wheel is spinning and how fast the rover is actually moving. If all six wheels have high average slip — above 40% — and the rover’s forward speed is below 15 centimetres per second, and both conditions hold for at least 15 consecutive steps — that is 0.6 seconds of wall-clock time — then the entrapment flag is set to 1.

The secondary signal is torque anomaly: if average motor torque exceeds 85% of the maximum limit AND torque is rapidly changing, we flag a torque anomaly. This rarely fires in practice because both conditions must hold simultaneously for 20 steps — we now know this dual-condition gate is too strict, and we plan to relax it.

The right diagram shows the full dual-brain architecture we envision for deployment. A CNN-GRU classifier — Phase 1 of our roadmap, not yet implemented — monitors sensor streams and outputs a probability distribution over three states: normal, sinking, and fully entrapped. Based on this, a decision switch activates either the Navigation Brain from RLRoverLab for normal traversal, or our Escape Brain — the GRU-PPO policy — for recovery. Once escape is confirmed, control returns to the Navigation Brain.”

What is CNN-GRU? (For Phase 1)

Note: The CNN-GRU Entrapment Detector is a planned future component (Phase 1). Here is what it means:

- **CNN part:** A 1D Convolutional Neural Network that slides a small window over the time-series of sensor data (e.g. 50 timesteps = 5 seconds at 10 Hz). It extracts local temporal patterns — like “slip was rising steeply here” or “torque had a sharp spike”
- **GRU part:** After the CNN extracts features from the local window, a GRU processes them sequentially to capture longer-range temporal dependencies — e.g. “slip was high for several consecutive windows” indicating genuine entrapment vs. a momentary obstacle

- **Output:** A 3-class probability over (normal, sinking, entrapped). We want $P(\text{entrapped}) > 0.5$ to trigger the Escape Brain
- **Target:** F1 score > 0.90 on the “entrapped” class, inference < 5 ms on RPi5
- **Training data:** collected by running the already-trained escape policy and logging simulation ground-truth states

SLIDE 9: Curriculum Learning & Training Pipeline

What is on this slide

- **Left:** Curriculum formula $d_{\text{sinkage}}(p) = d_{\text{min}} + p \cdot (d_{\text{max}} - d_{\text{min}})$ with $d_{\text{min}} = 0.12$ m, $d_{\text{max}} = 0.22$ m, $p \in [0, 1]$
- **Left:** curriculum_milestones.png — the actual training data plot (see slide description below)
- **Right:** PPO hyperparameter table, hardware table, domain randomization box

curriculum_milestones.png — what this plot shows

- X-axis: training timestep (0 to 200K)
- Y-axis (left): curriculum progress p from 0 to 1
- The curve shows curriculum progress rising from 0 and saturating at 1.0 around step 110K, then staying flat
- This means: by 110K steps, the agent was good enough at shallow sinkage (12 cm) that the scheduler promoted it to the full 22 cm sinkage range
- After saturation, all 64 environments sample sinkage uniformly between 12 and 22 cm every episode reset

What to say

“Curriculum learning is one of the most important design choices in our training. Without it, the agent would face 22-centimetre sinkage immediately, which is a very hard problem to learn from scratch. Instead, we start at 12 cm and increase progressively.

The formula is simple: sinkage depth equals minimum depth plus progress times the range. Progress is a scalar between 0 and 1 that the curriculum scheduler controls based on the agent’s recent success rate.

The plot on the left shows actual training data. Curriculum progress rises to 1.0 at around step 110,000 — this is a positive result. It means the agent was solving the shallow sinkage problem well enough to be promoted to full difficulty.

The hyperparameter table on the right shows the key training settings. Note the entropy coefficient of 0.03 — this is important because it prevents the policy from collapsing to one action too early. And the GRU sequence length of 16 means during each update we feed the GRU 16 consecutive steps at a time for backpropagation through time.”

Key numbers:

- Curriculum saturated at $\sim 110\text{K}$ steps out of 200K
- 64 parallel environments
- Wall time: 6–8 hours for 200K steps on RTX 5070 Ti
- Domain randomization: motor gain $\mathcal{U}(0.8, 1.2)$, obs noise $\mathcal{N}(0, 0.02)$, spawn yaw $\pm 30^\circ$

SLIDE 9b: DRL Training Loop Block Diagram

What is on this slide

A flowchart showing the complete PPO-GRU training loop:

- **Curriculum Scheduler** (top left): reads config, updates sinkage depth (dashed blue arrow) to the Stochastic Environment, and updates reward weights
- **Stochastic Environment** (orange box): Newton MPM + MuJoCo, 64 envs. Sends state $\mathbf{s}_t$ down to PPO, receives action $\mathbf{a}_t$ back
- **Reward function** (middle): 7-component reward v4. Receives (s_t, a_t) from environment, sends r_t to PPO
- **PPO box** (right): contains Actor (GRU) and Critic (GRU) sub-boxes, plus a loss box (PPO clip loss + value loss + entropy loss, Adam optimizer, $\text{lr} = 3 \times 10^{-4}$)
- **Experience sequence** (bottom): stores trajectories (s_t, a_t, r_t, s_{t+1}) in a replay pool, samples minibatches for PPO updates

What to say

“This diagram shows one iteration of the training loop. The environment generates a state, the actor network samples an action, the environment responds with the next state, and the reward function computes the 7-component reward. All of this is stored in the experience sequence.

When 24 rollout steps have accumulated, PPO takes a minibatch, computes the clipped policy loss, the value function loss, and an entropy bonus, and updates the actor and critic weights together using Adam.

The curriculum scheduler runs in parallel, monitoring whether the agent is succeeding, and adjusting the sinkage depth and reward weights accordingly. This is the dashed blue feedback loop.”

SLIDE 10: Training Overview — training_overview.png

What is on this slide

- **Large left plot:** training_overview.png — a multi-panel training dashboard
- **Right stat boxes:** 3 key numbers: $\sim 35\%$ (0.5 m milestone hit rate), 7.8 (mean episode reward, last 10%), 1.0 (curriculum progress reached at $\sim 110\text{K}$)
- A highlight box: “Policy std dev: 1.303, still exploring; 2M+ steps planned”

training_overview.png — what the panels show

This is a dashboard of multiple training metrics plotted against timestep (0–200K):

- **Mean episode reward:** starts near 0, rises to approximately 7–8 in the final phase. Consistently positive throughout — good sign that the policy is not doing catastrophically wrong things.
- **0.5 m milestone hit rate:** rises from 0% to approximately 35% by the end. This is the primary escape success metric.
- **1.0 m milestone hit rate:** approximately 15%. Lower than 0.5 m because fewer episodes last long enough.
- **Curriculum progress:** rises sharply, flattens at 1.0 around 110K.
- **Policy std dev:** rises from 1.0 to 1.303. Rising std dev indicates the policy is still in an exploratory phase — it has not settled on a fixed strategy. More training is needed for convergence.

What to say

“These three numbers summarise our training results at the 200K timestep mark.

35% escape rate at the 0.5-metre milestone. 7.8 average episode reward. Curriculum progress at 1.0 — complete.

The training dashboard on the left shows these metrics evolving over time. The reward is consistently positive, which means the policy is never in a regime of completely random or catastrophically bad behaviour.

The policy standard deviation of 1.303 is actually telling us the policy is still exploring — it has not yet converged to a specific escape strategy. This is expected at 200K steps. Bi & Ding used 4 million steps to get 90%. We plan to scale to 2 million steps with a larger GPU cluster.”

SLIDE 11: Escape Progress & Curriculum — escape_rate.png

escape_rate.png — what it shows

- X-axis: timestep (0 to 200K)
- Y-axis (left): escape success rate at 0.5 m milestone (0 to 50%)
- Y-axis (right): mean x-displacement per episode (in metres)
- The 0.5 m hit rate rises from near 0 to approximately 35%, showing clear learning progress
- The mean displacement also rises, showing the rover is consistently travelling further from its starting position
- The 1.5 m milestone curve stays at 0 throughout — this is expected because the episode resets after 0.5 m is reached (this is not a failure; it means the 0.5 m milestone consistently terminates the episode before 1.5 m can be reached)

curriculum_milestones.png — what it shows

- X-axis: timestep (0 to 200K)
- Y-axis: curriculum progress p (0 to 1), and sinkage depth in cm (12 to 22)
- A sharp rise from 0 to 1.0 between steps 0 and 110K, then flat
- May include vertical milestone markers at certain timesteps

What to say

“The escape rate plot on the left shows the 0.5-metre milestone hit rate rising clearly over time. 35% is not a failure — it is a baseline for a problem that nobody has solved before. With 10 times more training steps, we expect this to approach Bi & Ding’s 90%. The 1.5-metre line stays at zero because episodes reset automatically when the rover clears 0.5 metres. This is correct behaviour by design.

The curriculum plot on the right shows a clean progression. The fact that it reaches 1.0 at 110K steps means the agent genuinely learned to escape shallow sinkage before being promoted to harder depths. The curriculum did its job.”

SLIDE 12: Loss & Policy Metrics

losses.png — what it shows

Three loss curves plotted against timestep:

- **Policy loss:** starts near 0, drifts gradually negative from -0.02 to approximately -0.08 . A slowly decreasing policy loss is a healthy sign — the policy is consistently improving.
- **Value loss:** spikes sharply around timestep 25K (when the curriculum introduces harder terrain), then decreases toward 0. A converging value loss means the critic is learning to predict returns accurately.
- **Entropy loss:** slowly decreasing. This means the policy is gradually concentrating — reducing action randomness — but not collapsing to zero (the entropy coefficient 0.03 prevents this).

policy_metrics.png — what it shows

- **Policy std dev:** rises from 1.0 to approximately 1.303 over 200K steps. Rising std dev at this stage indicates the policy is expanding its action distribution — exploring new strategies as the curriculum becomes harder.
- **Episode length:** oscillates between 200 and 600 steps. This variance indicates the rover is solving some episodes quickly (fast escape) and spending longer in others (slower or harder terrain). Both outcomes contribute to learning.

What to say

“The loss plots on the left tell us the training is technically healthy. Value loss spikes at 25K steps: that is exactly when the curriculum made the terrain harder, and the critic temporarily lost its accuracy — then recovered. Policy loss drifts slightly negative throughout, which means policy gradient updates are consistently pointing in the improvement direction. Entropy loss declines without going to zero, meaning the policy is concentrating without collapsing.

The policy standard deviation rising to 1.3 might look alarming, but it is actually expected at this stage: as the terrain gets harder, the policy needs to explore more diverse strategies. At convergence — which we expect around 2 million steps — this should decrease below 1.0.”

SLIDE 12b: Reward Breakdown — reward_breakdown.png

reward_breakdown.png — what it shows

- Stacked or multi-line plot of all 7 reward components vs. timestep
- r_{escape} is the dominant positive signal by a large margin (weight 20)
- r_{progress} is consistently small but positive (the rover is moving forward, just slowly)
- r_{rocking} is positive when the entrapment flag is active (early in episodes)
- p_{slip} is a small negative signal post-escape
- p_{tilt} is very small (the rocker-bogie suspension inherently keeps the rover stable)
- p_{smooth} is very small (the policy learned smooth actions quickly)
- $p_{\text{abnormal}} \approx 0$ throughout — this is the known limitation (dual-condition gate too strict)

detection_flags.png — what it shows

- Two binary time-series (or rate plots): the entrapment flag activation rate and the torque anomaly flag activation rate
- Entrapment flag is always 1 at episode start (by design) and decreases as the rover escapes
- Torque anomaly flag fires rarely ($< 1\%$ of steps), confirming the known issue

SLIDE 13: Escape Behaviour Demo — env_metrics.png

env_metrics.png — what it shows

- Environment-level metrics: mean wheel slip per episode, mean forward velocity per episode, mean episode displacement
- Shows the rover’s motion improving over time as training progresses
- Slip rates may start high and decrease slightly as the policy learns more efficient wheel usage

What to say

“The simulation demo slide shows emergent escape behaviours. Four strategies emerged completely without explicit programming: rocking back and forth to pack sand under the wheels; differential steering to yaw toward areas with better traction; coordinated steer-and-drive to escape diagonally; and progressive acceleration to avoid sinking again after initial escape.

The environment metrics plot confirms the rover is achieving forward displacement and managing wheel slip throughout training.”

Emergent strategies explained

- **Rocking:** the policy alternates between positive and negative ω_{drive} , creating a back-and-forth motion that compacts loose sand and creates a firmer base for the next forward push. This is incentivised by r_{rocking} .
- **Differential steering:** by pointing some wheels in different directions, the rover creates a yawing torque that can shift weight to wheels with better ground contact
- **Diagonal escape:** combining steer angle with drive creates a path that is not straight, allowing the rover to find a direction with less resistance
- **Progressive acceleration:** rather than spinning wheels at full speed (which digs deeper), the policy applies gradual torque increases

SLIDE 14: Training Results Summary

summary_table.png — what it shows

- A visual dashboard summarising all key metrics side-by-side
- Possibly bar charts or a formatted table of: reward, escape rates, curriculum progress, mean velocity, policy std dev

What to say

“Our training summary table shows where we are against targets. Four out of six metrics are already in a good state: reward is positive, curriculum is complete, mean forward velocity exceeds 10 cm/s, and value loss has converged. The two metrics still in progress are escape rate and policy std dev — both require more training steps.

The most important planned next step is scaling to 2 million timesteps on a larger GPU, which we believe will push escape rate above 50%.”

SLIDE 16: Future Work Roadmap

What is on this slide

Three priority columns in a grid:

- **Priority 1 (Now):** Scale to 2M+ steps; CNN-GRU sinkage detector ($F1 > 0.90$); GAIL reward fusion ($0.9R_H + 0.1R_D$); ablation study (GRU vs MLP, curriculum on/off)

- **Priority 2 (Sim-to-Sim):** Transfer to MuJoCo/Gazebo; sloped terrain $\pm 15^\circ$; lunar gravity stress test; 5-seed statistical reproducibility
- **Priority 3 (Sim-to-Real):** ONNX export to RPi5 (< 5 ms); sand tray test; outdoor gravel validation; dual-brain integration with RLRoverLab

What to say

“Our roadmap has three priority levels, gated sequentially. Priority 1 is immediate: scale to 2 million steps and build the CNN-GRU entrapment detector. Priority 2 is sim-to-sim validation: can the policy transfer to a different simulator, or on sloped terrain, without retraining? Priority 3 is the physical test: export to ONNX, run on the Raspberry Pi 5, and test in a real sand tray.

Each priority is conditional on the previous one succeeding. We are realistic about the contingency.”

SLIDE 17: Contributions

What to say

“To summarise, our five technical contributions are:

First, the first Newton MPM plus Isaac Lab integration for a rover entrapment simulation under Martian gravity.

Second, a novel dual-sensor detection scheme using slip ratio and torque anomaly without any external sensing.

Third, a recurrent GRU-PPO policy that uses memory to discover multi-step escape strategies.

Fourth, a seven-component shaped reward with curriculum-driven sinkage progression.

Fifth, a complete end-to-end pipeline from physics simulation to RL training to evaluation dashboard.

All five of these together represent the first DRL entrapment recovery system for a 6-wheel rocker-bogie rover in physics-accurate granular simulation.”

EXPECTED QUESTIONS & MODEL ANSWERS

Q: Why GRU and not LSTM?

A: GRU has 2 gates versus LSTM’s 3 gates plus a separate cell state. This gives us roughly 30% fewer parameters, and inference on the Raspberry Pi 5 takes under 5 ms with GRU versus around 7 ms with LSTM. More importantly, our episodes are 200–600 steps at 25 Hz — 8 to 24 seconds of physical time — and GRU performs equivalently to LSTM for sequences shorter than about 5 seconds. LSTM’s cell state provides an advantage mainly for very long sequences with slow-varying dependencies, which we do not have here. PPO’s clipping already stabilises the gradient flow, so LSTM’s additional gating is redundant. We chose GRU as a principled efficiency-first decision.

Q: Why GRU and not a Transformer?

A: Transformers use self-attention over the full context window, which scales quadratically with sequence length. At 25 Hz with sequences of 16, this is tractable, but transformers require storing and attending over the full trajectory history — challenging on an embedded system. More importantly, transformers have no natural recurrence: during online deployment, you would need to cache the full history. A GRU naturally maintains a fixed-size hidden state, making it ideal for real-time deployment. Several recent works show that for low-dimensional proprioceptive control (no images, no point clouds), GRUs match transformer performance while being substantially cheaper to deploy.

Q: Why not a feedforward (MLP) policy?

A: A feedforward policy receives only the current observation with no memory. The rover cannot directly observe its sinkage depth or soil composition. With no history, the policy cannot distinguish between “I have been slipping for 15 steps and am genuinely trapped” and “I just hit a momentary obstacle.” The GRU hidden state implicitly encodes this temporal context. The comparison table on slide 12 shows the GRU has sinkage tracking capability that the feedforward baseline lacks.

Q: Your escape rate is only 35%. Is this good?

A: For the problem we are solving — physics-accurate MPM granular terrain, Mars gravity, 6-wheel rocker-bogie, 200K training steps with no expert demonstrations — yes, 35% is a meaningful baseline result. The closest comparable work, Bi & Ding, achieves 90% but with 20 times more training, rigid terrain, Earth gravity, and expert-demonstrated GAIL rewards. If we normalise for training budget, our rate is proportionally competitive. We need approximately 2 million steps to reach convergence.

Q: What is MPM and why is it better than a rigid terrain?

A: MPM stands for Material Point Method. It models granular media (sand, soil) as a continuum using particles that carry mass and deformation history, processed on a background grid each timestep. The Drucker-Prager yield criterion determines when soil flows plastically versus compresses elastically. Rigid terrain cannot capture three key phenomena: progressive wheel sinkage as sand is compressed and displaced; bulldozing resistance as sand piles in front of a moving wheel; and particle redistribution during wheel spin, which is what actually entraps a rover. Without these effects, any trained policy will fail in reality.

Q: What is POMDP and why does it matter?

A: A Partially Observable Markov Decision Process is an MDP where the agent cannot observe the full system state. In our case, the rover’s sensors cannot directly measure sinkage depth, soil compaction, or the exact extent of entrapment. It only sees 29 numbers. The GRU

hidden state acts as a belief state, accumulating information over time to infer the unobserved quantities. Without the recurrent memory, the agent would have to make decisions with severely incomplete information.

Q: Why 64 parallel environments?

A: PPO requires large batches of on-policy experience to compute stable policy gradient estimates. With 64 environments each contributing 24 rollout steps per update, we collect 1,536 transitions per PPO iteration. This gives us good diversity — different sinkage depths, different spawn orientations, different trajectory segments — reducing the variance of the gradient estimate. Fewer environments would require longer rollouts (risking stale data) or more iterations with noisier gradients.

Q: What is GAIL and why did Bi & Ding use it but you did not?

A: GAIL — Generative Adversarial Imitation Learning — trains a discriminator to distinguish between expert demonstrations and the policy’s own trajectories. The discriminator’s output becomes an additional reward signal, pulling the policy toward expert-like behaviour. Bi & Ding used it because their robot has a simple 4-wheel skid-steer platform and they could physically demonstrate escape manoeuvres. For Martian regolith in a 6-wheel rocker-bogie with Mars gravity, there are no existing expert demonstrations. We had to design our reward function entirely from physical principles, which is a harder problem but does not require expert data.

Q: Why did you choose PPO and not SAC, TD3, or DDPG?

A: PPO is the standard choice for locomotion and manipulation tasks with recurrent policies in Isaac Lab because: (1) it is on-policy, which works well with short rollouts and dense rewards; (2) its clipped objective provides training stability without careful hyperparameter tuning of target networks or replay buffers; (3) PPO with a recurrent policy (GRU) has well-understood BPTT semantics; (4) it parallelises cleanly across many environments. SAC and TD3 are off-policy and require a replay buffer, which complicates the GRU hidden state management (you need to store and replay hidden states alongside transitions). This is solvable but adds engineering complexity without clear benefit for our scenario.

Q: What is the Drucker-Prager model?

A: Drucker-Prager is a yield criterion for granular materials, analogous to Von Mises yield for metals. It says: if the stress state in the material falls inside a cone in stress space (defined by friction angle and cohesion), the material behaves elastically. If the stress reaches the cone surface, the material yields and flows plastically. For dry sand on Mars (friction angle $\sim 30^\circ$, negligible cohesion), this accurately captures how sand flows around and under a sinking wheel. The model is calibrated by the friction coefficient $\mu = 0.9$ in our simulation.

Q: How does two-way coupling work between MPM and MuJoCo?

A: Two-way coupling means information flows in both directions. MuJoCo computes the rigid body state (positions, velocities) of the rover. The wheel surfaces are represented as Signed Distance Functions (SDF), which define the shape of the wheel in a mathematical form that the MPM solver can query. At each timestep: (1) the SDF is passed to the MPM solver, which computes how sand particles should be displaced by the wheel surface; (2) the resulting sand impulse forces are fed back to MuJoCo as external forces on the wheel. This loop runs 4 times per physics step (4 substeps). Without two-way coupling, the rover would not sink because the sand would have no way to exert resistance force.

Q: What does the Raspberry Pi 5 deployment involve?

A: We export the trained GRU-PPO policy to ONNX format (a hardware-neutral neural network format). On the RPi5, we run the ONNX Runtime library to execute inference. The input is a 29-dimensional observation vector, the output is a 10-dimensional action. At 10 Hz policy frequency, we have 100 ms per step; our target is < 5 ms inference time, which is well within budget. The GRU hidden state is maintained between steps as a 256-dimensional vector stored in memory. No GPU is needed.

Q: Why did you not test on the physical rover?

A: The physical AAU Mars Rover is hardware maintained by Aalborg University in Denmark. We do not have physical access at this stage of the project. Our roadmap places physical testing in Priority 3, contingent on successful sim-to-sim validation in Priority 2. This is standard practice in sim-to-real research: validate in simulation first, then transfer.

Q: What does your torque anomaly detector do, and why does it not fire?

A: The torque anomaly fires only when *both* conditions hold simultaneously for 20 steps: average motor torque exceeds 85% of the rated maximum, AND torque is changing rapidly ($\Delta > 0.10$). The policy learned to avoid saturating motors — possibly because the slip penalty already discourages high-torque spinning. The dual-condition gate is too strict. We plan to relax it: use OR instead of AND, lower the torque threshold to 70%, shorten the window to 10 steps, or replace it with a continuous soft penalty proportional to average torque utilisation.

Q: What is the observation space exactly?

A: The 29-dimensional observation consists of: 6 wheel angular velocities (one per wheel), 6 wheel slip ratios, 4 steering joint positions, 3 IMU accelerations (x, y, z in body frame), 1 gravity z-component (useful for detecting tilt), 6 drive torques (one per wheel), 1 entrapment flag (f_{ent}), 1 torque anomaly flag (f_{anom}), and 1 normalised x-displacement (how far the rover has moved from its start position). Total: $6+6+4+3+1+6+1+1+1 = 29$.

Q: What is the action space?

A: The 10-dimensional action consists of: 6 target angular velocities for the drive wheels (range ± 6 rad/s), and 4 target angles for the steering joints (range ± 0.6 rad). Actions are sampled from a Gaussian distribution during training; the mean is used directly during evaluation. This is continuous control with per-wheel independence, enabling asymmetric strategies like spinning one side forward and the other backward.

Q: What is sim-to-real gap and how do you address it?

A: Sim-to-real gap is the difference in performance between simulation and the physical world, caused by unmodelled dynamics, sensor noise, actuator imperfections, and environmental variation. We address it through domain randomisation: motor gains are randomised $\pm 20\%$, observation noise $\mathcal{N}(0, 0.02)$ is added, and spawn yaw is randomised $\pm 30^\circ$. This forces the policy to be robust to parameter uncertainty. In Priority 2, we will also test transfer to different simulators (MuJoCo, Gazebo) to measure cross-simulator generalisation.

Q: Could you use imitation learning instead of RL?

A: Imitation learning requires demonstrations of the correct escape behaviour. For a 6-wheel rover in granular Martian regolith, such demonstrations do not exist. We cannot manually teleoperate the physical escape because: (a) we lack a physical testbed with Martian soil properties, and (b) the 4–24 minute signal delay on Mars makes real-time teleoperation impossible. RL from reward signals is the only feasible approach.

Q: What happens if the rover tips over?

A: The environment does not currently model full capsizing. The tilt penalty p_{tilt} (weight 0.3) penalises angular velocity around the x and y axes, discouraging roll and pitch. The rocker-bogie suspension also mechanically limits tilt through its passive joints. If the rover were to fully tip, the episode would reset. In future work, we plan to add an explicit episode termination condition for excessive tilt and a corresponding terminal penalty.

QUICK-REFERENCE NUMBERS CARD

Print this and keep it in front of you during the presentation.

Metric	Value
Spirit entrapment duration	8 months (recovery attempts)
Earth–Mars signal latency	4–24 min one-way
Earth–Mars distance	150M km (at entrapment)
Rover wheels	6 (rocker-bogie)
Drive joints	6, ± 6 rad/s
Steer joints	4, ± 0.6 rad
Action dimensions	10
Observation dimensions	29
Martian gravity	3.72 m/s^2
Earth gravity	9.81 m/s^2
Sand bed size	$2 \times 2 \times 0.25 \text{ m}$
MPM particles per env	38,400
Voxel size	0.05 m
Wheel–sand friction	$\mu = 0.9$
Parallel environments	64
Physics frequency	50 Hz
Policy frequency	25 Hz
Total timesteps	200,000
Wall time	6–8 hours
Sinkage curriculum	12 cm $\rightarrow$ 22 cm
Curriculum saturation	$\sim 110\text{K}$ steps
GRU hidden size	256
FC layers	128 $\rightarrow$ GRU $\rightarrow$ 64
BPTT sequence length	16
PPO clip ϵ	0.2
Discount γ	0.99
GAE λ	0.95
Learning rate	3×10^{-4}
Rollout length	24 steps
Entropy coeff	0.03
0.5 m escape rate	$\sim 35\%$
1.0 m escape rate	$\sim 15\%$
Mean episode reward	7.8
Policy std dev (final)	1.303
Bi & Ding escape rate	90% (4M steps, rigid, Earth)
RPi5 inference target	$< 5 \text{ ms}$
Policy frequency (deploy)	10 Hz
CNN-GRU F1 target	> 0.90 (“entrapped” class)

PRESENTATION TIMING GUIDE

Assuming a 15-minute presentation plus 5-minute Q&A:

Section	Time
Title + Motivation (slides 1, 2, 2b)	2.5 min
Objectives + Related Work (slides 3, 4, 4b)	2.5 min
System Design (slides 5b, 5c, 5d, 6)	3.0 min
RL Formulation + Architecture (slides 7, 7b)	2.5 min
Detection + Curriculum (slides 8, 9, 9b)	1.5 min
Results (slides 10, 11, 12, 12b, 13, 14)	2.0 min
Future Work + Contributions (slides 16, 16b, 17)	1.0 min
Total	15 min

If you are running long: Cut slide 2b (extended Spirit timeline) and slide 9b (DRL training loop block diagram) — they are the most detail-heavy and the content is captured in adjacent slides.

If you are asked to go longer: Spend more time on slide 7b (network architecture) and slide 6 (MPM physics stack) — these are the most technically impressive parts.